

Casos de Prueba

Plataforma Voz del Ciudadano

CP-01 — Registro con DNI inválido *RF relacionado: RF-01*

Precondición Ninguna. El sistema está disponible.

Entrada DNI: 1234567 (7 dígitos), correo: test@mail.com, contraseña: pass1234

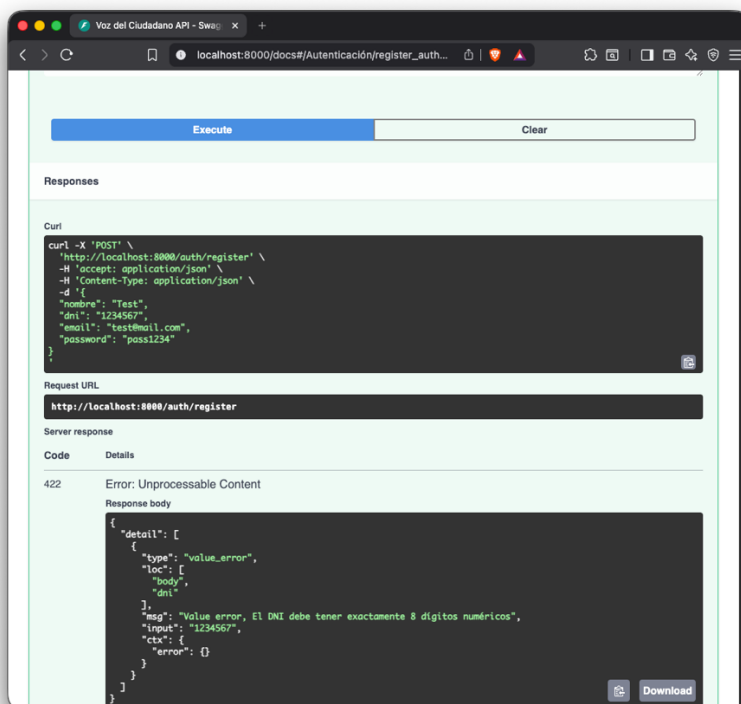
Resultado esp. HTTP 422 — mensaje: “El DNI debe tener exactamente 8 dígitos numéricos”

Estado Pasa

Código:

```
def test_cp01_registro_dni_invalido():
    """DNI con 7 dígitos debe retornar error de validación (422)."""
    r = client.post("/auth/register", json={
        "nombre": "Test", "dni": "1234567",
        "email": "cp01@test.com", "password": "pass1234"
    })
    assert r.status_code == 422
    assert "DNI" in r.text
```

CP-01 → captura del 422 en Swagger al enviar DNI inválido (debe ser 8 dígitos):



CP-02 — Doble firma sobre la misma propuesta *RF relacionado: RF-03*

Precondición Usuario registrado. Propuesta activa. Usuario ya firmó una vez.

Entrada Segunda solicitud POST /proposals/{id}/sign con el mismo token.

Resultado esp. HTTP 409 — mensaje: “Ya has firmado esta propuesta anteriormente”

Estado Pasa

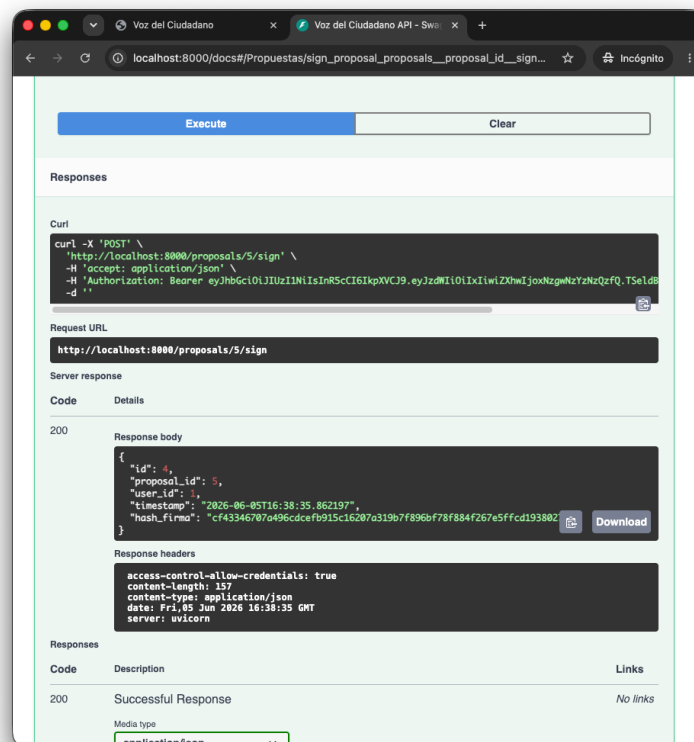
Código:

```
def test_cp02_doble_firma():
    """Un mismo usuario no puede firmar la misma propuesta dos veces (409)."""
    token_autor = _registrar("20000001", "autor_cp02@test.com")
    token_firma = _registrar("20000002", "firma_cp02@test.com")
    pid = _crear_propuesta(token_autor)

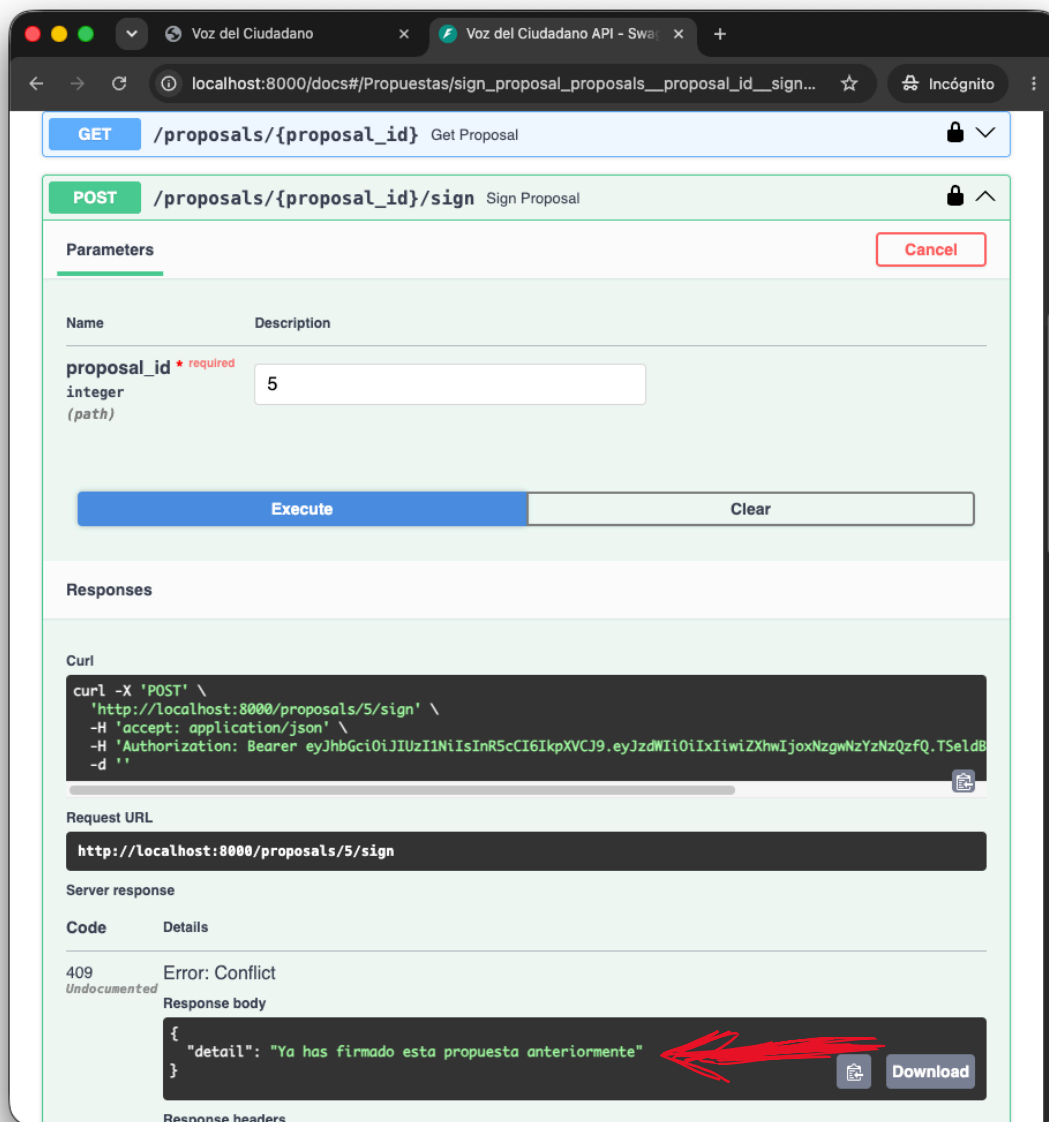
    r1 = client.post(f"/proposals/{pid}/sign", headers=_auth(token_firma))
    assert r1.status_code == 200

    r2 = client.post(f"/proposals/{pid}/sign", headers=_auth(token_firma))
    assert r2.status_code == 409
    assert "Ya has firmado" in r2.json()["detail"]
```

CP-02 → captura del 409 al intentar firmar dos veces:



Ejecutamos de nuevo deberia salir error 409:



CP-03 — Firma sin autenticación *RF relacionado: RF-03*

Precondición Propuesta activa existente. Sin token JWT en la cabecera.

Entrada POST /proposals/{id}/sign sin cabecera Authorization.

Resultado esp. HTTP 401 — acceso denegado.

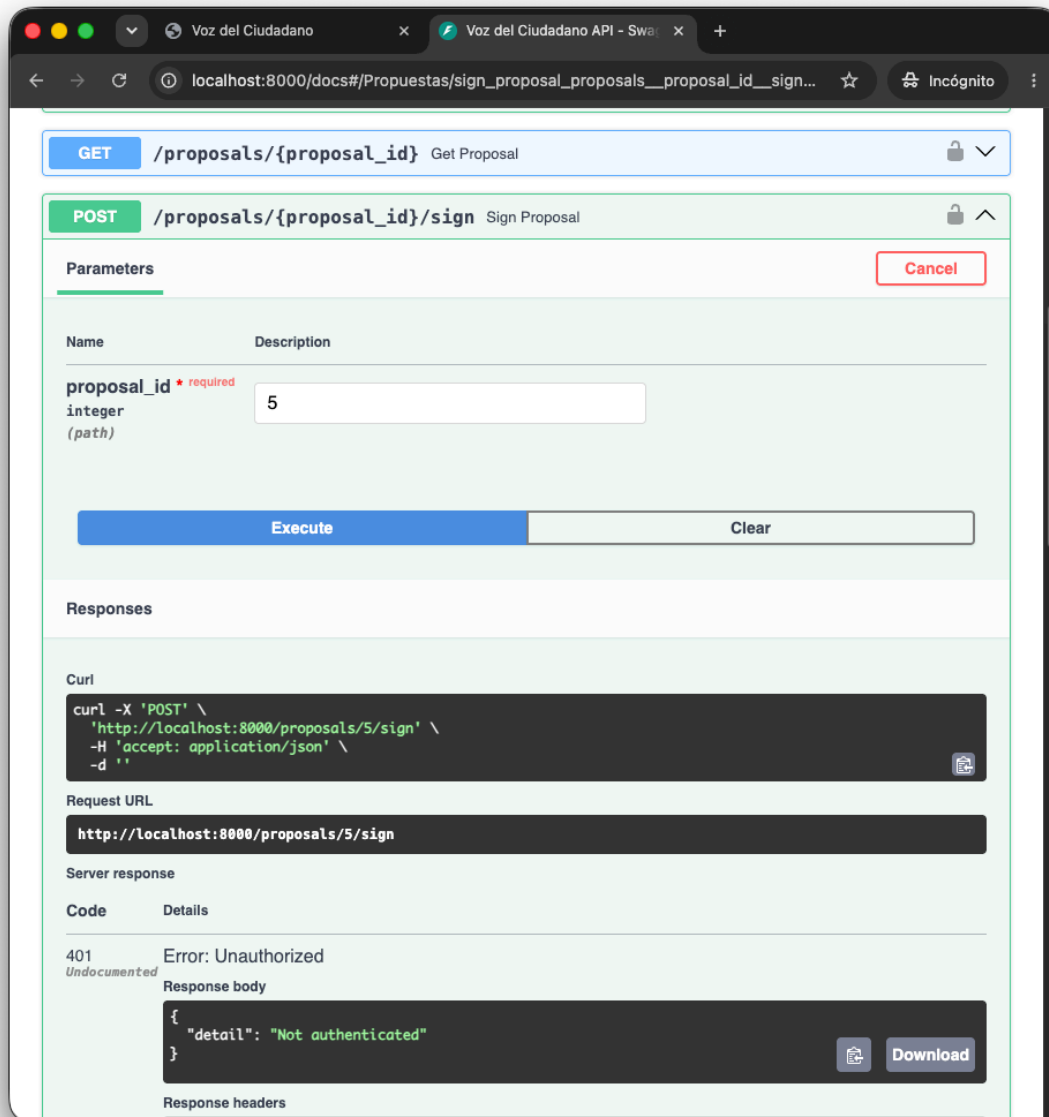
Estado Pasa

Codigo:

```
def test_cp03_firmar_sin_autenticacion():  
    """Firmar sin token debe retornar 401 Unauthorized."""  
    token = _registrar("30000001", "cp03@test.com")  
    pid = _crear_propuesta(token)
```

```
r = client.post(f"/proposals/{pid}/sign")
assert r.status_code == 401
```

CP-03 → captura del 401 al firmar sin token:



CP-04 — Congelamiento automático al alcanzar el límite *RF relacionado: RF-04 – RF-05*

Precondición Propuesta activa. Límite configurado en 1 firma (entorno de prueba).

Entrada Un usuario distinto al autor firma la propuesta.

Resultado esp. Estado cambia a enviada, hash_congelamiento de 64 caracteres hexadecimales generado, tracking_congreso registrado.

Estado Pasa

Codigo:

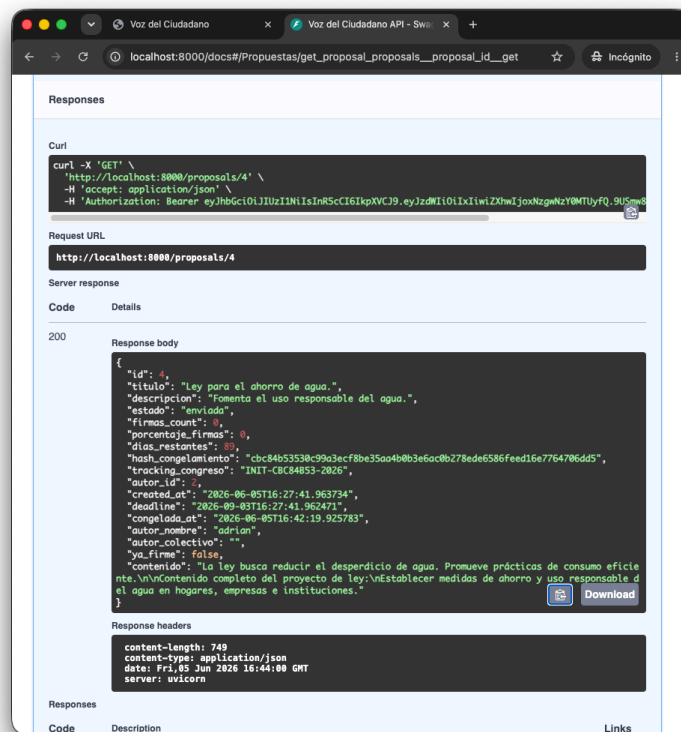
```
def test_cp04_congelamiento_automatico(monkeypatch):
    """Al alcanzar el límite de firmas la propuesta se congela y genera hash SHA-256."""
    import patterns.facade as facade_module
    monkeypatch.setattr(facade_module, "FIRMA_LIMITE", 1)

    token_autor = _registrar("40000001", "autor_cp04@test.com")
    token_firmante = _registrar("40000002", "firmante_cp04@test.com")
    pid = _crear_propuesta(token_autor)

    client.post(f"/proposals/{pid}/sign", headers=_auth(token_firmante))

    r = client.get(f"/proposals/{pid}")
    data = r.json()
    assert data["estado"] in ("congelada", "enviada")
    assert data["hash_congelamiento"] is not None
    assert len(data["hash_congelamiento"]) == 64 # SHA-256 hex
```

CP-04 → captura del response con estado: enviada y el hash generado



- "estado": "enviada"
- "hash_congelamiento": "cbc84b53..." (64 caracteres)
- "tracking_congreso": "INIT-CBC84B53-2026"

Ejecución del test:

```
darcyortegasullcaccori@Darcys-MacBook-Air backend % python3 -m pytest tests/test_app.py -v
===== test session starts =====
=====
platform darwin -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0 -- /Library/Frameworks/Python.framework/Versions/3.13/bin/python3
cachedir: .pytest_cache
rootdir: /Users/darcyortegasullcaccori/Documents/DS_PC3/backend
plugins: anyio-4.13.0
collected 4 items

tests/test_app.py::test_cp01_registro_dni_invalido PASSED
[ 25%]
tests/test_app.py::test_cp02_doble_firma PASSED
[ 50%]
tests/test_app.py::test_cp03_firmar_sin_autenticacion PASSED
[ 75%]
tests/test_app.py::test_cp04_congelamiento_automatico PASSED
[100%]

===== warnings summary =====
.....
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/starlette/formparsers.py:12
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/starlette/formparsers.py:12:
PendingDeprecationWarning: Please use `import python_multipart` instead.
    import multipart

tests/test_app.py::test_cp02_doble_firma
tests/test_app.py::test_cp02_doble_firma
tests/test_app.py::test_cp02_doble_firma
tests/test_app.py::test_cp03_firmar_sin_autenticacion
tests/test_app.py::test_cp03_firmar_sin_autenticacion
tests/test_app.py::test_cp04_congelamiento_automatico
tests/test_app.py::test_cp04_congelamiento_automatico
tests/test_app.py::test_cp04_congelamiento_automatico
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/sqlalchemy/sql/schema.py:3596: DeprecationWarning:
datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent
datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return util.wrap_callable(lambda ctx: fn(), fn) # type: ignore

tests/test_app.py::test_cp02_doble_firma
tests/test_app.py::test_cp02_doble_firma
tests/test_app.py::test_cp02_doble_firma
```

```
tests/test_app.py::test_cp03_firmar_sin_autenticacion
tests/test_app.py::test_cp04_congelamiento_automatico
tests/test_app.py::test_cp04_congelamiento_automatico

/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/jose/jwt.py:311: DeprecationWarning:
datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent
datetimes in UTC: datetime.datetime.now(datetime.UTC).

    now = timegm(datetime.utcnow().utctimetuple())

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 4 passed, 15 warnings in 2.88s
=====

darcyortegasullcaccori@Darcys-MacBook-Air backend %
```